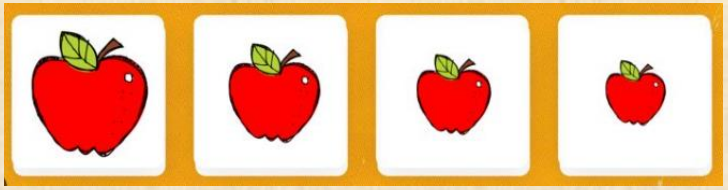
Lecture 4

Complexity (leftovers)s
Sorting
HeapSort, MergeSort & QuickSort

Reading: Chapters 7-8

(depends on the book edition)



Solving Recursive Equations
by Repeated Substitution

Consider *BSearch* on input of size n

$$T(n) \le T\left(\frac{n}{2}\right) + c$$

$$\le T\left(\frac{n}{4}\right) + c + c = T\left(\frac{n}{4}\right) + 2c$$

$$\le T\left(\frac{n}{2^3}\right) + c + 2c = T\left(\frac{n}{2^3}\right) + 3c$$

...

$$\le T\left(\frac{n}{2^j}\right) + jc$$

...

$$\le T\left(\frac{n}{2^{\log n}}\right) + c \log n$$

$$= T(1) + c \log n = b + c \log n \in O(\log n)$$

not a formal proof

b: number of steps
for base case: $T(1)$
c: bound on number of steps
in each iteration

Solving Recursive Equations by Induction

- The informal substitution method tells us what the inductive claim should be.

BSearch: $T(n) = O(\log n)$

- Complete proof: General n
- $T(n) \le T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c$

$k \le \log n + 1$

Claim: if $2^{k-1} \le n < 2^k$, then $T(n) \le b + ck$

Proof of claim: by induction on k for $2^{k-1} \le n < 2^k$

- Base case: $k = 1$, $T(2^0) = T(1) = b$
- Inductive assumption for $1, \ldots, k$: if $2^{k-1} \le n < 2^k$ then $T(n) \le b + ck$
- Induction step for $k+1$, let $2^k \le n < 2^{k+1}$

  so $2^{k-1} \le \left\lfloor \frac{n}{2} \right\rfloor < 2^k$ and $2^{k-2} \le \left\lfloor \frac{n}{2} \right\rfloor - 1 < 2^k$

  $$T(n) \le \max\left\{T\left(\left\lfloor \frac{n}{2} \right\rfloor\right), T\left(\left\lfloor \frac{n}{2} \right\rfloor - 1\right)\right\} + c \le b + ck + c = b + c(k+1)$$

- Since $k \le \log n + 1$, we get: $T(n) \le (b + c) + c \log(n+1)$

Tight Bound – binary search

- We proved $T(n) \in O(\log n)$
- We can also show that $T(n) \in \Omega(\log n)$
  - How?
  - Choose b' and c' and prove by induction that $T(n) \geq b' + c' \log_2 n$
  - same structure as the upper bound
- Corollary: $T(n) \in \Theta(\log n)$

Little-o
(Non-Tight Upper Bound)

$f(n) \in o(g(n))$ if for every constant $c > 0$ there exists n_0 such that for all $n > n_0$, $f(n) < cg(n)$

Equivalently, $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$

E.g.:

$2n = o(n^2)$

$\log n = o(n)$

but $n \notin o(2n)$

Little-ω (Non-Tight Lower Bound)

$f(n) \in \omega(g(n))$ if for every constant $c > 0$ there exists n_0 such that for all $n > n_0$, $f(n) > cg(n)$

Equivalently, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$

E.g. :

$n \log n = \omega(n)$,

$2^{0.001n} = \omega(n^{300})$

Summary

- Asymptotic notation:
  - O,o: upper bounds
  - Ω,ω: lower bounds
  - Θ: tight bound
- Enables complexity analysis in a machine-independent way by ignoring constant factors and focusing on large inputs
- Cases: best / worst / average
- Examples: bubble sort, binary search

This Lecture

- Comparison based sorting algorithms:
  - Heap sort
  - Merge sort
  - Quick sort

The Sorting Problem

Input: A sequence of n objects $<a_1, a_2, \ldots, an>$ with order relation $\leq$ defined on them

Output: A permutation (reordering) $<b_1, b_2, \ldots, b_n>$ of the input sequence satisfying $b_1 \leq b_2 \leq \cdots \leq b_n$

Why Sort?

- Allows:
  - Binary search in $\Theta(\log n)$ time worst case
  - $O(1)$ time to access k^{th} largest element in the array for any k
  - Easy to detect duplicate values

Evaluation of Sorting Algorithms

- Time
- Space
- Stability

Time Complexity

- How many steps are required?
  - Trivial lower bound: $\Omega(n)$
  - Simple upper bound: $O(n^2)$ check each element against every other element (achieved by Bubble Sort, Insertion sort)
  - The big question: Can we close the gap between these two bounds

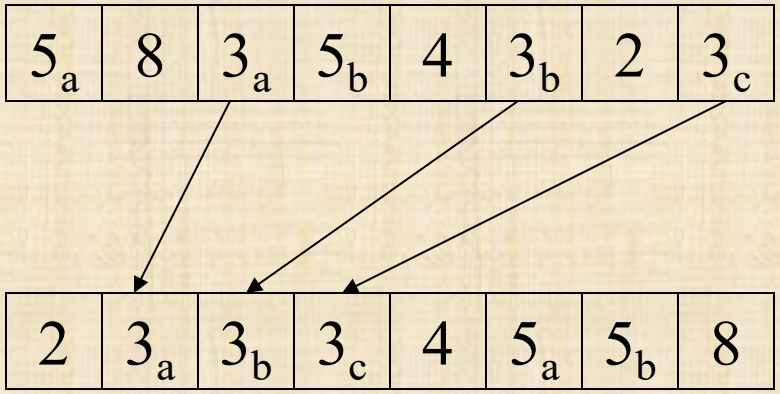
Space Complexity

- The space required to sort *n* items.
  - Is copying needed?
    $O(n)$ additional space
  - In-place sorting – no copying
    $O(1)$ additional space
  - Somewhere in between for "temporary" or recursion stack, e.g., $O(\log n)$ space

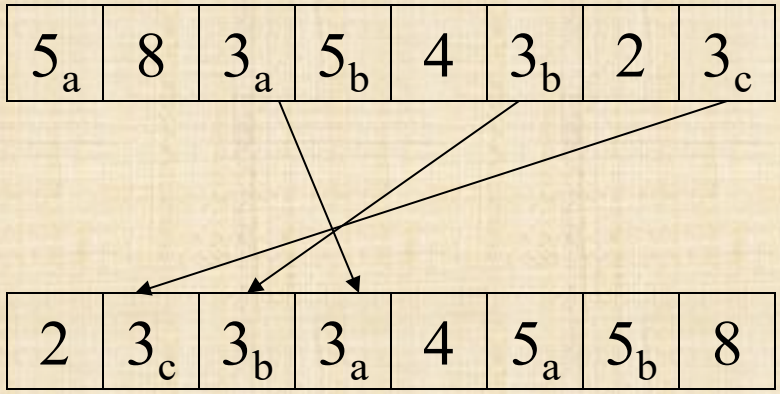
Stability

- A *sorting* algorithm is *stable* if it does not change the relative order of duplicate values
- Extremely important property for databases
- Example:
  - Sort by name and then by country
  - Are the items still sorted by name within each country?

Example – (un)stable sort



Stable Sort



Unstable Sort

Example – stable sort

Last Name	First Name
York	Paola
Ayers	Julio
York	Samson
Allen	Bryan
Decker	Triston
Stafford	Jaquan
Choi	Jada
Beard	Paola
Ayers	Gonny
Flynn	Taliyah
York	Breanna
Choi	Paola

Input L_0

Last Name	First Name
York	Breanna
Allen	Bryan
Ayers	Gonny
Choi	Jada
Stafford	Jaquan
Ayers	Julio
York	Paola
Beard	Paola
Choi	Paola
York	Samson
Flynn	Taliyah
Decker	Triston

L_1: the input L_0 sorted by first name

Last Name	First Name
Allen	Bryan
Ayers	Gonny
Ayers	Julio
Beard	Paola
Choi	Jada
Choi	Paola
Decker	Triston
Flynn	Taliyah
Stafford	Jaquan
York	Breanna
York	Paola
York	Samson

A stable sort of L_1 by last name

Last Name	First Name
Allen	Bryan
Ayers	Julio
Ayers	Gonny
Beard	Paola
Choi	Jada
Choi	Paola
Decker	Triston
Flynn	Taliyah
Stafford	Jaquan
York	Samson
York	Paola
York	Breanna

A non-stable sort of L_1 by last name

Example – stable sort

Last Name	First Name
York	Paola
Ayers	Julio
York	Samson
Allen	Bryan
Decker	Triston
Stafford	Jaquan
Choi	Jada
Beard	Paola
Ayers	Gonny
Flynn	Taliyah
York	Breanna
Choi	Paola

Input L_0

Last Name	First Name
York	Breanna
Allen	Bryan
Ayers	Gonny
Choi	Jada
Stafford	Jaquan
Ayers	Julio
York	Paola
Beard	Paola
Choi	Paola
York	Samson
Flynn	Taliyah
Decker	Triston

L_1: the input L_0 sorted by first name

Last Name	First Name
Allen	Bryan
Ayers	Gonny
Ayers	Julio
Beard	Paola
Choi	Jada
Choi	Paola
Decker	Triston
Flynn	Taliyah
Stafford	Jaquan
York	Breanna
York	Paola
York	Samson

A stable sort of L_1 by last name

Last Name	First Name
Allen	Bryan
Ayers	Julio
Ayers	Gonny
Beard	Paola
Choi	Jada
Choi	Paola
Decker	Triston
Flynn	Taliyah
Stafford	Jaquan
York	Samson
York	Paola
York	Breanna

A non-stable sort of L_1 by last name

Bubble Sort: Properties

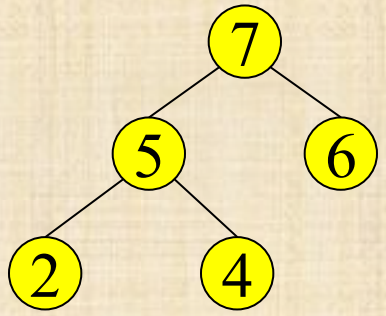
- Time complexity:
  - Worst case: $\Theta(n^2)$
  - Best case: $\Theta(n)$
  - Average: $\Theta(n^2)$
- Space:
  - in place
- Stable:
  - Yes (a[i], a[i+1] are swapped only if a[i]>a[i+1])

5_a	1	2	5_b	4	3_b	2	3_c

1	2	5_a	5_b	4	3_b	2	3_c

Heap Sort

- We use a Max-Heap for n objects:
  - Root node = *A[1],*
  - Children of *A[i] = A[2i], A[2i+1]*
- Keep track of current heap size s
  - First round: $s = n$



value	7	5	6	2	4
index	1	2	3	4	5

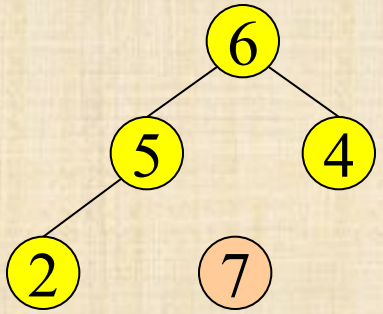
$s = n = 5$

A: the input and output array
n: the number of elements to sort
s: the number of element in the heap

Basic Operation

- Invoke *DeleteMax*
- This reduces the heap size, s, by 1
- Store the deleted max element immediately after the last element of the heap (i.e., in $A[s+1]$)
- Invariants:
  - $A[s+1]$ ... $A[n]$ are sorted in increasing order
  - $A[1]$ ... $A[s]$ form a max heap

value	6	5	4	2	7
index	1	2	3	4	5

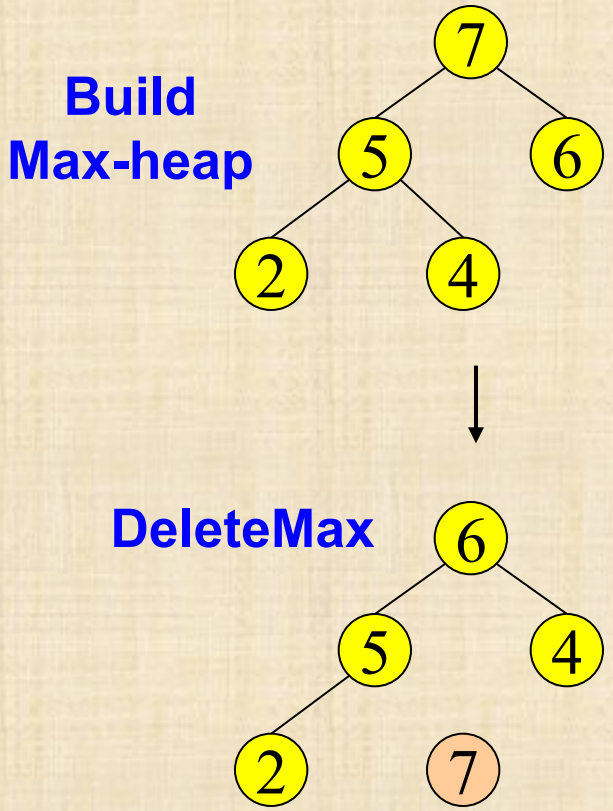


$s = 4$

Heap Sort

value	6	5	4	2	7

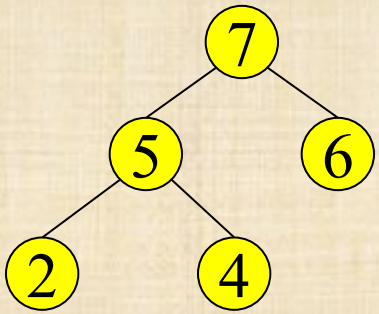
```
HeapSort(A)
    BuildMaxHeap(A)
    n = number of elements in A
    for s = n downto 2
          x=A[1];
          PercDown(1, A[s], s-1, A)
          A[s]=x;
```



Repeated DeleteMax

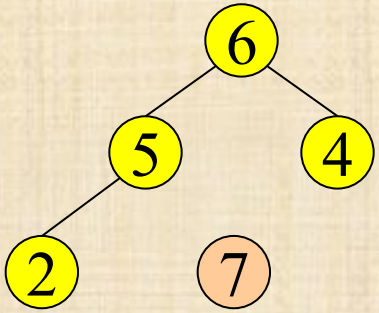
7	5	6	2	4
1	2	3	4	5

$s = n = 5$



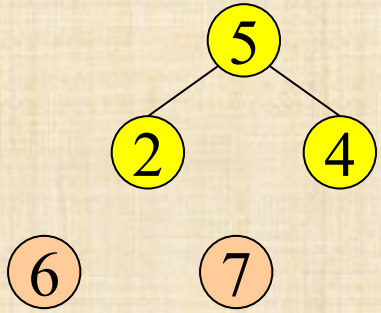
6	5	4	2	7
1	2	3	4	5

$s = 4$



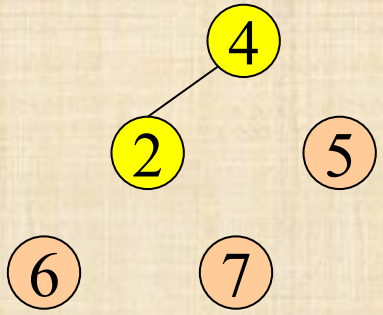
5	2	4	6	7
1	2	3	4	5

$s = 3$



4	2	5	6	7
1	2	3	4	5

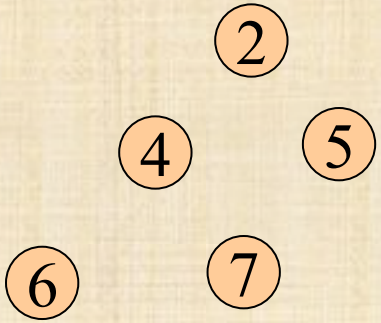
$s = 2$



Repeated DeleteMax

- After $n - 1$ DeleteMax:
  - The size of the heap, $s = 1$
  - The array is sorted

index	1	2	3	4	5
value	2	4	5	6	7



- Recall:
  - $A[s+1]..A[n]$ are sorted
  - $A[1]$ ... $A[s]$ form a max heap

HeapSort: Time Analysis

```
HeapSort(A)
    BuildMaxHeap(A)
    n=length(A)
    For s=n downto 2
        x=A[1];
        PercDown(1, A[s], s-1, A)
        A[s]=x;
```

HeapSort: Time Analysis

```
HeapSort(A)
    BuildMaxHeap(A)
    n=length(A)
    For s=n downto 2
        x=A[1];
        PercDown(1, A[s], s-1, A)
        A[s]=x;
```

Build MaxHeap $O(n)$

$(n-1)$ DeleteMax : $O(n \log n)$

DeleteMax is $O(\log s)$

- Worst case upper bound:

$$O(n) + O(n \log n) = O(n \log n)$$

HeapSort: Time Analysis

Worst case: $\Theta(n \log n)$

- Upper bound: $O(n \log n)$
- To show that the worst case is $\Omega(n \log n)$:
  - Consider an input sorted from largest to smallest

HeapSort Properties

- Time complexity:
  - Worst case: $\Theta(n \log n)$
  - Best case:
    - $\Theta(n)$ (all elements are identical)
    - $\Theta(n \log n)$ (for distinct elements – *proof omitted*)
  - Average case:
    - $\Theta(n \log n)$ – *proof omitted*
- Space complexity:
  - In Place
- Not stable

Merge Sort

A sorting algorithm based on divide and conquer

Divide and Conquer: A strategy for solving a problem by dividing it to sub-problems and combining the results into a solution to the original problem

Divide and Conquer

- **Divide** into a number of sub-problems
  (similar to the original problem but smaller)
- **Conquer** solve the sub-problems (recursively)

  if a sub-problem is small enough,
  just solve it in a straightforward manner
- **Combine** the solutions to the sub-problems into the solution for the original problem

Merge Sort

- **Divide** *n*-element sequence into 2 subsequences, each with *n/2* elements
- **Conquer**: Sort the two subsequences recursively **using merge sort**
- **Combine**: merge the two sorted subsequences to produce the sorted answer
- **Recursion base case**: a sequence with 1 element is sorted.

Recursive Merge Sort

```
Mergesort(A, p, r)
if p < r then
      q ← ⌊(p+r)/2⌋
      Mergesort(A,p,q)
      Mergesort(A,q+1,r)
      Merge(A,p,q,r)
```

A: the input and output array
A[p...r]: subarray
p,r: first and last indices of the subarray in A

To sort an array call **`Mergesort(A, 1, length(A))`**.

Merge

```
Merge(A,p,q,r)
n1← q-p+1, n2← r-q          %sizes of the
                            %subarrays
L[1,..,n1]← A[p,...,q]       % copy A from p to q
                             % into a new array L
R[1,..,n2]← A[q+1,...,r]    % copy A from q+1 to r
                             % into a new array R
L[n1+1] ←∞,  R[n2+1] ←∞
i←1,  j←1
for k ← p to r do
      if L(i)≤R(j)
      then A[k] ← L[i]
i←i+1

      else A[k] ← R[j]
                j←j+1
```

Insert the merged values into A

A: the input and output array
A[p..q], A[q+1..r] : subarrays
p,q,r: indexes

Time complexity $\Theta(n)$

Merge Sort – Time complexity

$T(n)$

```
Mergesort(A, p, r)
if p < r then
        q ← ⌊(p+r)/2⌋
        Mergesort(A,p,q)
        Mergesort(A,q+1,r)
        Merge(A,p,q,r)
```

O(1)

$2 \cdot T(\frac{n}{2})$

O(n)

$$T(n) \leq 2T\left(\frac{n}{2}\right) + c_1 n$$

$$T\left(\left\lceil\frac{n}{2}\right\rceil\right) + T\left(\left\lfloor\frac{n}{2}\right\rfloor\right) \approx 2T\left(\frac{n}{2}\right)$$

* Like we did in lecture 3 (for Binary Search) we ignore rounding issues

Analyzing Recursive Programs

1. Express the running time $T(n)$ as a recursive equation or inequality
2. Solve the recursive equation
   - For an upper-bound, you can bound $T(n)$ by a simpler larger quantity
   - For a lower-bound, you can bound $T(n)$ by a simpler smaller quantity

Merge Sort Analysis

- Let $T(n)$ be the running time of MergeSort on an array of n elements
- **Divide**: $\Theta(1)$
- **Conquer**:
  - Two recursive calls, each $T(n/2)$
- **Combine**: merge in $\Theta(n)$
- **The base case (n=1)**: $\Theta(1)$
- **The general case (n>1)**:

$$T(n) \leq 2T(n/2) + c_1 n$$

Merge sort Analysis
Upper Bound

$$T(n) \le 2T\left(\frac{n}{2}\right) + c_1 n$$

$$\le 2\left(2T\left(\frac{n}{4}\right) + c_1\frac{n}{2}\right) + c_1 n$$

$$= 4T\left(\frac{n}{4}\right) + 2c_1 n$$

$$\le$$

$$\vdots$$

$$\le 2^k T\left(\frac{n}{2^k}\right) + kc_1 n$$

$$\vdots$$

$$\le 2^{\log n} T(1) + (\log n)\, c_1 n$$

$$\le bn + c_1 n \log n \in O(n \log n)$$

There is an induction hiding here

Assuming n is a power of 2

$T(1) \le b$

MergeSort $O(n \log n)$

Theorem: If $n = 2^k, k \geq 1, T(n) \leq 2T\left(\frac{n}{2}\right) + c_1 n$ and $T(1), T(2) \leq c_2$, for some constants $0 \leq c_1, c_2$ then $T(n) \in O(n \log n)$

Proof: We need to show that there exists d such that for any k, $T(2^k) \leq d 2^k \log 2^k$ for some $d > 0$.

We denote $d = \max(c_1, c_2)$.
Since $n = 2^k, k \geq 1$ we use induction on k

- Base case ($k = 1$):
  $T(2^1) = T(2) = c_2 \leq d \cdot 2^1 \cdot \log 2^1$ because $d \geq c_2$

Proof cont.

- Induction hypothesis:
  for $1 \le k' \le k, T(2^k) \le d2^k \log 2^k$
- Induction step:

$$T(n) = T(2^{k+1}) \le 2T\left(\frac{2^{k+1}}{2}\right) + c_1 2^{k+1}$$

$$\le 2T(2^k) + d2^{k+1} \qquad \text{for } d \ge \max\{c_1, c_2\}$$

$$\le 2d2^k \log(2^k) + d2^{k+1} \qquad \text{by ind. hyp.}$$

$$= d2^{k+1}(\log(2^k) + 1)$$

$$= dn\left(\log\left(\frac{n}{2}\right) + \log 2\right) = dn(\log n - \log 2 + \log 2)$$

$$= dn(\log n)$$

MergeSort - $O(n \log n)$

For general n

Theorem: If $T(n) \leq T\left(\left\lceil \frac{n}{2} \right\rceil\right) + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c_1\, n$, and $T(1), T(2) \leq c_2$, then $T(n) \in O(n \log n)$.

Proof : We need to show that there exists some $d > 0$ such that for any n, $T(n) \leq dn\lceil \log n \rceil$

We denote $d \geq \max\{c_1, c_2\}$

We continue by induction on $k = \lceil \log n \rceil$

- Base case: $k = 1$

  $T(2^1) = T(2) = c_2 \leq d \cdot 2 \cdot \log 2$ since $d \geq c_2$

Proof cont.

- Induction hypothesis

  Assume that for any n s.t. $\lceil \log n \rceil < k$,

  $$T(n) \leq dn\lceil \log n \rceil$$

- Induction step: we prove for n s.t. $\lceil \log n \rceil = k$

$$T(n) \leq T\left(\left\lceil \frac{n}{2} \right\rceil\right) + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c_1 n$$

$$\leq T\left(\left\lceil \frac{n}{2} \right\rceil\right) + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + dn \quad (d \geq \max\{c_1, c_2\})$$

by ind. hyp. since $\left\lceil \log\left(\left\lceil \frac{n}{2} \right\rceil\right) \right\rceil < k$

$$\leq d\left\lceil \frac{n}{2} \right\rceil (k-1) + d\left\lfloor \frac{n}{2} \right\rfloor (k-1) + dn$$

$$= \mathrm{d}n(k-1) + dn = dnk = dn\lceil \log n \rceil$$

MergeSort

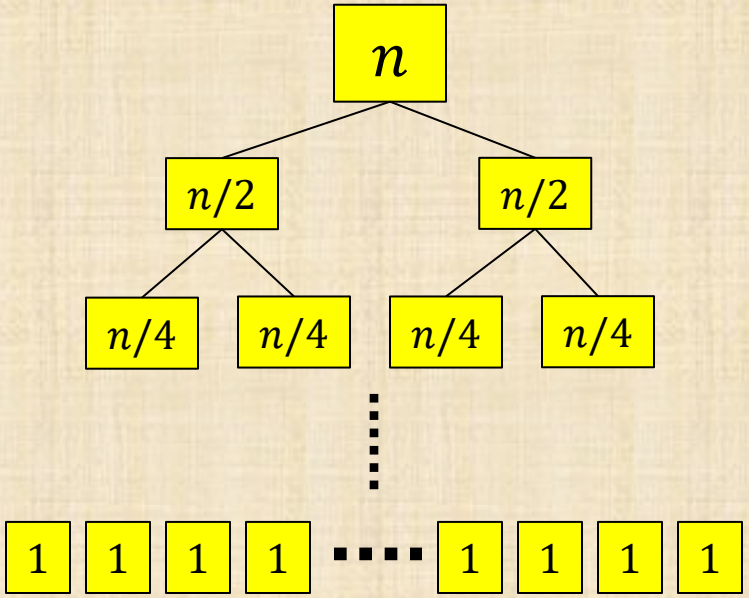
- Worst Case:
  - $T(n) \leq cn \log n \in O(n \log n)$
  - $T(n) \geq c'n \log n \in \Omega(n \log n)$
  - $T(n) \in \Theta(n \log n)$

← can be shown by a similar induction

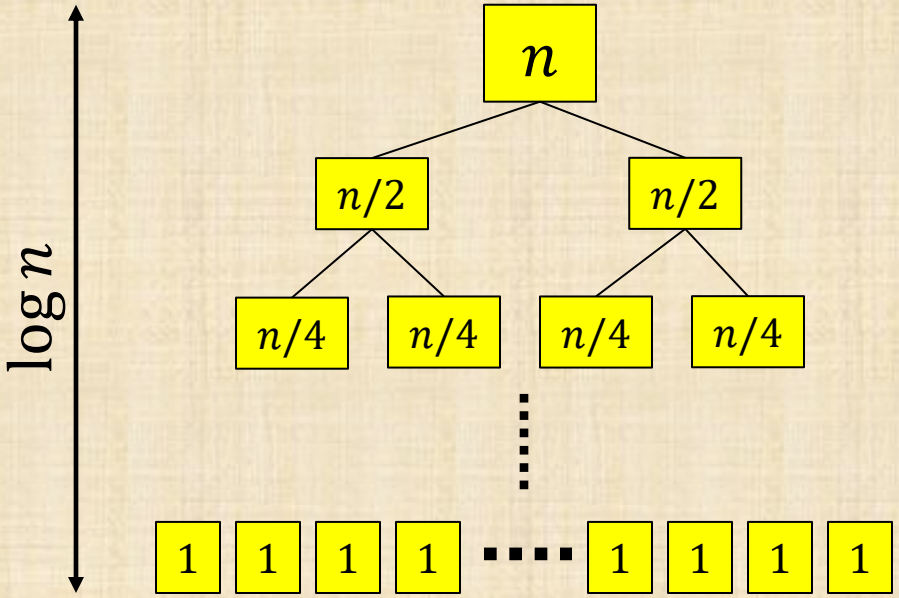
- Best case and Average case:

  $\Theta(n \log n)$ - since the number of operations of MergeSort only depends on the size of the input, not on the content of the input.

Recursion tree method (informal)



Recursion tree method (informal)



cost of divide+combine

$$\theta(n)$$

$$2 \cdot \theta(n/2) = \theta(n)$$

$$4 \cdot \theta(n/4) = \theta(n)$$

$$\vdots$$

$$\mathrm{n} \cdot \theta(1) \quad = \theta(n)$$

$$\theta(n \log n)$$

Variation of Recursive Merge Sort

- When the array is small: use bubble sort

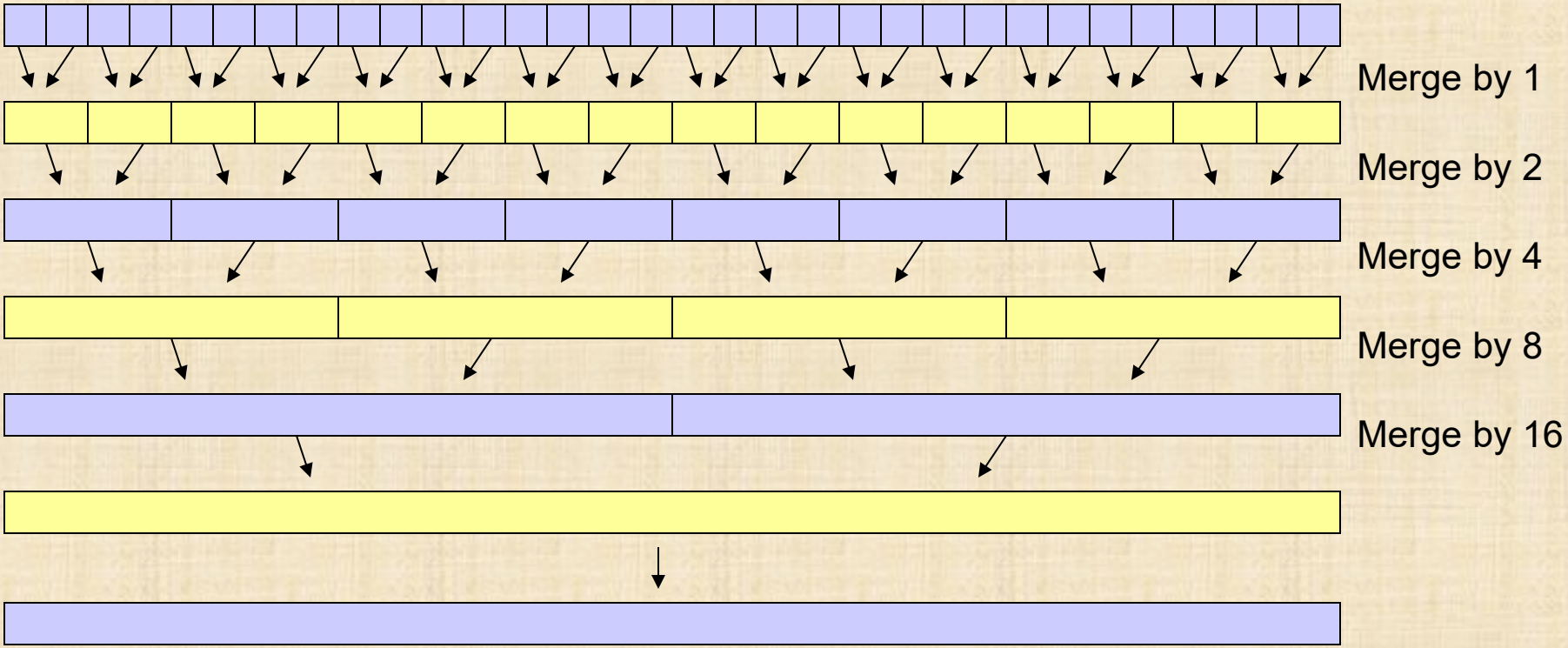
```
Mergesort(A, p, r)
if p < r-5 then
      q←⌊p+r)/2⌋
      Mergesort(A,p,q)
      Mergesort(A,q+1,r)
      Merge(A,p,q,r)
else
      buble_sort(A,p,r)
```

More than 6 elements

Time complexity ?

Iterative MergeSort

Reduces copying & recursive calls overhead



Properties of MergeSort

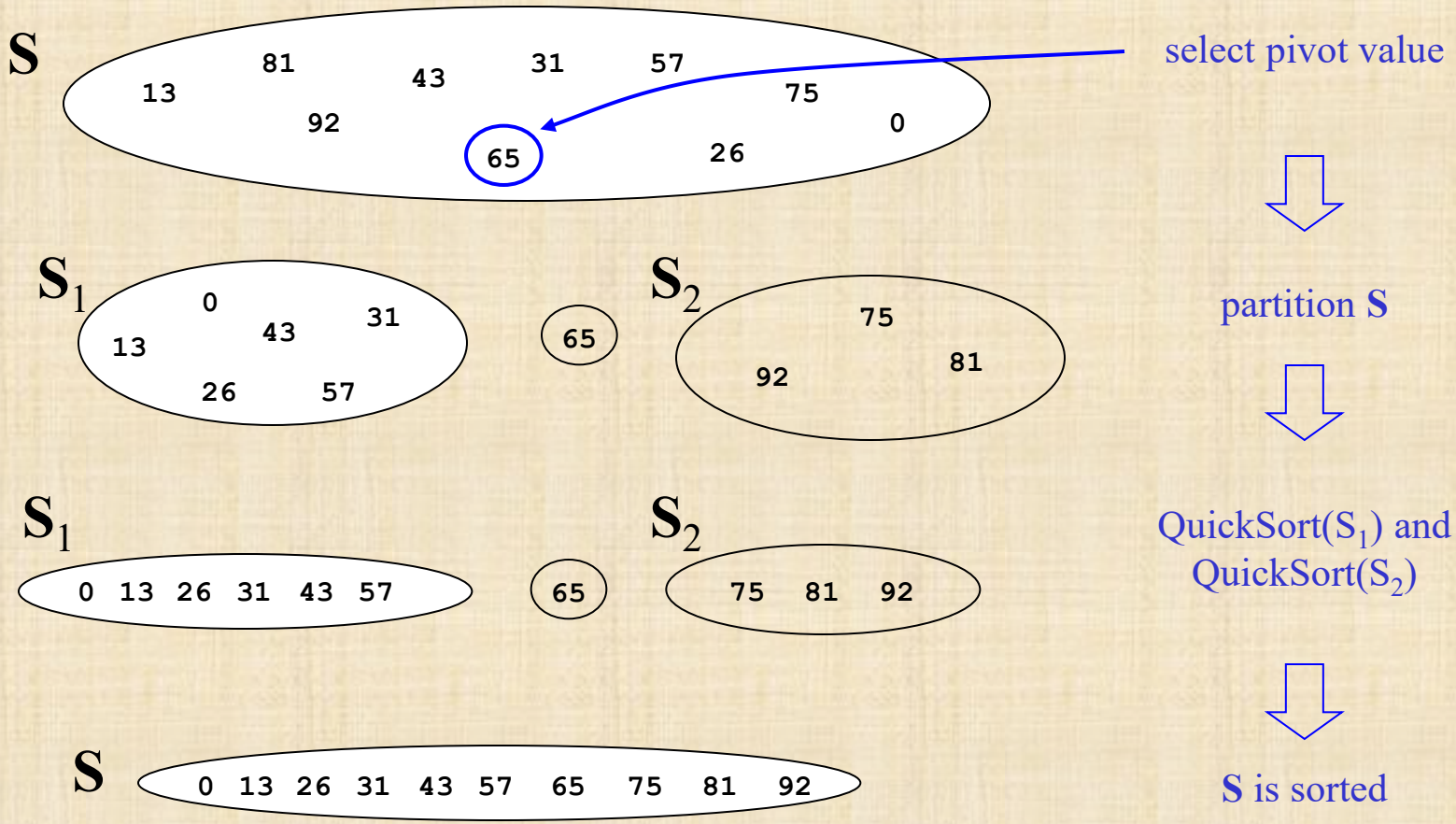
- In-place?
  - No - Requires an auxiliary array

    ($\Theta(n)$ extra space)
- Stable?
  - Yes – during merge, give precedence to lower index in case of ties.

Quicksort

- Uses a divide and conquer strategy
- In-place: does not require the $\theta(n)$ extra space (unlike MergeSort)

The Steps of QuickSort



Quicksort: General Idea

- **Divide:** Partition array into left and right sub-arrays
  - Select an element of the array, called pivot
  - Move to left sub-array all elements $\leq$ pivot
  - Move to right sub-array all elements $>$ pivot
- **Conquer:** Recursively sort the sub-arrays on the left and on the right of the pivot
- **Combine:** nothing to do
- **The base case:** an empty sub-array is sorted
  (or bubble-sort a small constant number of elements)

Recursive QuickSort

```
Quicksort(A, p, r)
if r-p > 0 then
      q ←Partition(A, p, r)
      Quicksort(A, p, q-1)
      Quicksort(A, q+1,r)
Return
```

To sort an array call ***Quicksort*(A, 1, length(A))**.

A: the input and output array
A[p..r]: subarray
p,r: first and last indexes

A Practical optimization

```
Quicksort(A, p, r)
if r-p > 20
      then q ←Partition(A, p, r)
           Quicksort(A, p, q-1)
           Quicksort(A, q+1,r)
      else SimpleSort(A,p,r)
```

More than 20 elements

To sort an array call ***Quicksort*(A, 1, length(A))**.

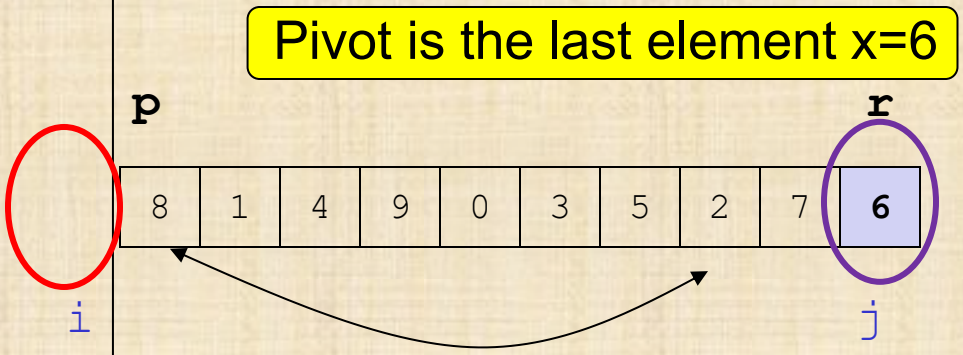
A: the input and output array
A[p..r]: subarray
p,r: first and last indexes

Details, details

- Picking the pivot
- Implementing the actual partitioning
- Dealing with cases where an element equals the pivot

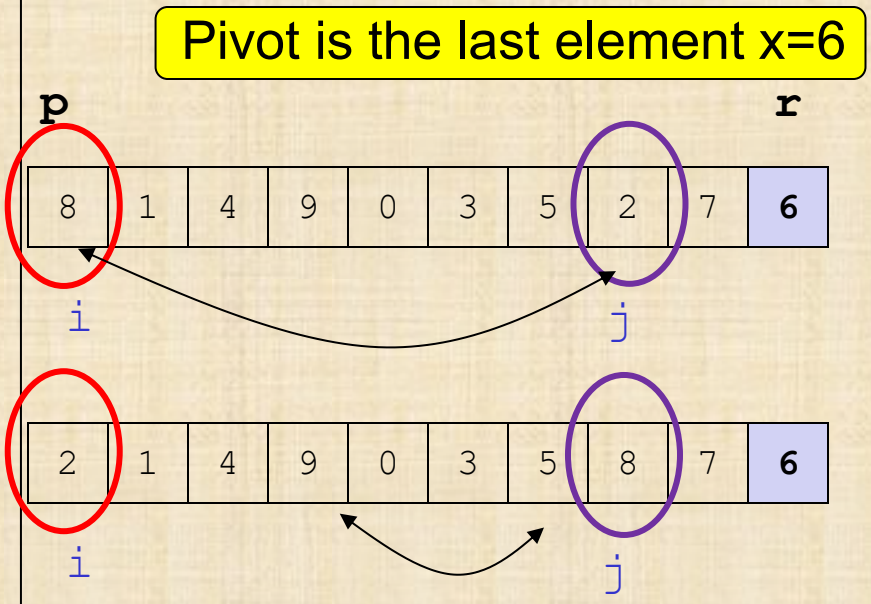
Partition

```
Partition(A,p,r)
x=A[r]; j=r; i=p-1;
While true
        repeat  j ← j - 1
        until A[j]≤x or j<p

        repeat i ← i +1
        until A[i]>x or i>r

        if i<j then
            A[i]↔A[j] % swap
        else
            A[j+1] ↔ A[r]
            return j+1
```



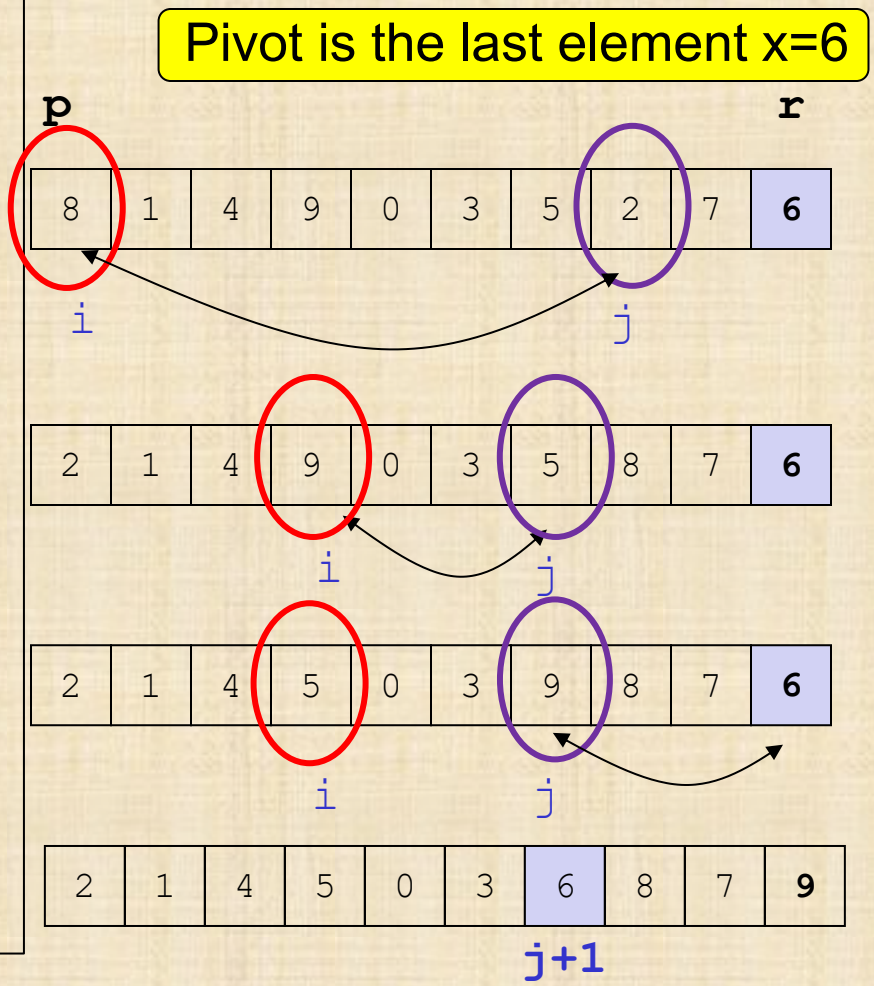
Partition

```
Partition(A,p,r)
x=A[r]; j=r; i=p-1;
While true
        repeat  j ← j - 1
        until A[j]≤x or j<p

        repeat i ← i +1
        until A[i]>x or i>r

        if i<j then
            A[i]↔A[j] % swap
        else
            A[j+1] ↔ A[r]
            return j+1
```



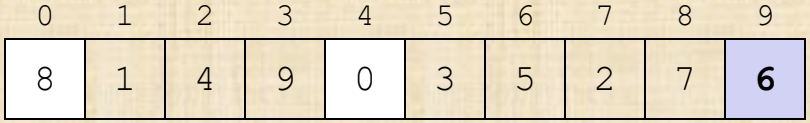
Partition

```
Partition(A,p,r)
x=A[r]; j=r; i=p-1;
While true
        repeat  j ← j - 1
        until A[j]≤x or j<p

        repeat i ← i +1
        until A[i]>x or i>r

        if i<j then
            A[i]↔A[j] % swap
        else
            A[j+1] ↔ A[r]
            return j+1
```



Partition

- Choose the pivot: the last element



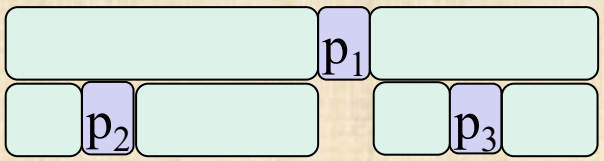
- Given the pivot, make one pass:
  - Set pointers *i* and *j* to *start* and *end* of array
  - Decrement *j* until you hit element $A[j] \leq pivot$
  - Increment *i* until you hit element $A[i] > pivot$
  - Swap *A[i]* and *A[j]*
  - Repeat until *i* and *j* cross

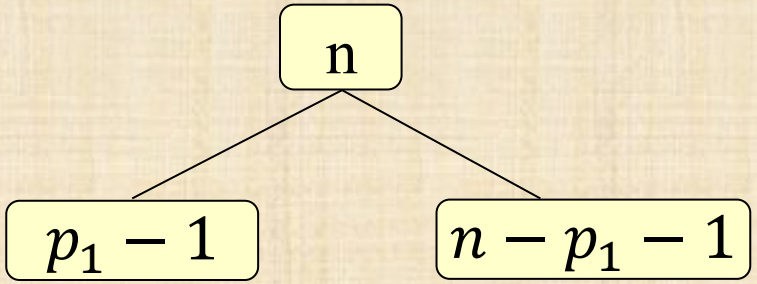
QuickSort Analysis

- The size of the two subarrays depends on the pivot
- Best case, worst case, average case

Quick Sort

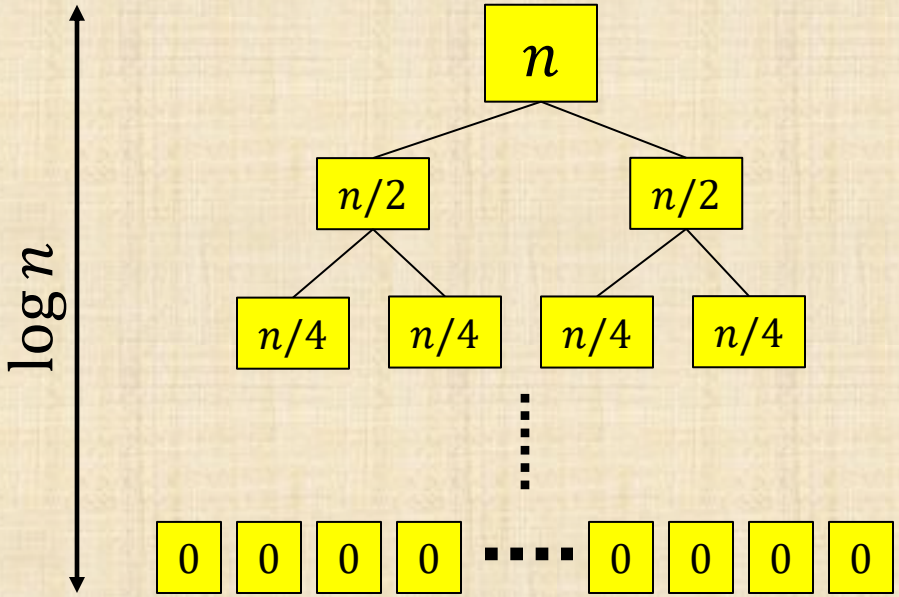
- The recursive complexity function:
  - $T(0) = O(1)$,
  - Let p be the location of the pivot after the partition
  - $T(n) = \Theta(n) + T(p-1) + T(n-p-1)$





QuickSort Best Case

The algorithm always chooses the best pivot – the one which produces a balanced partition

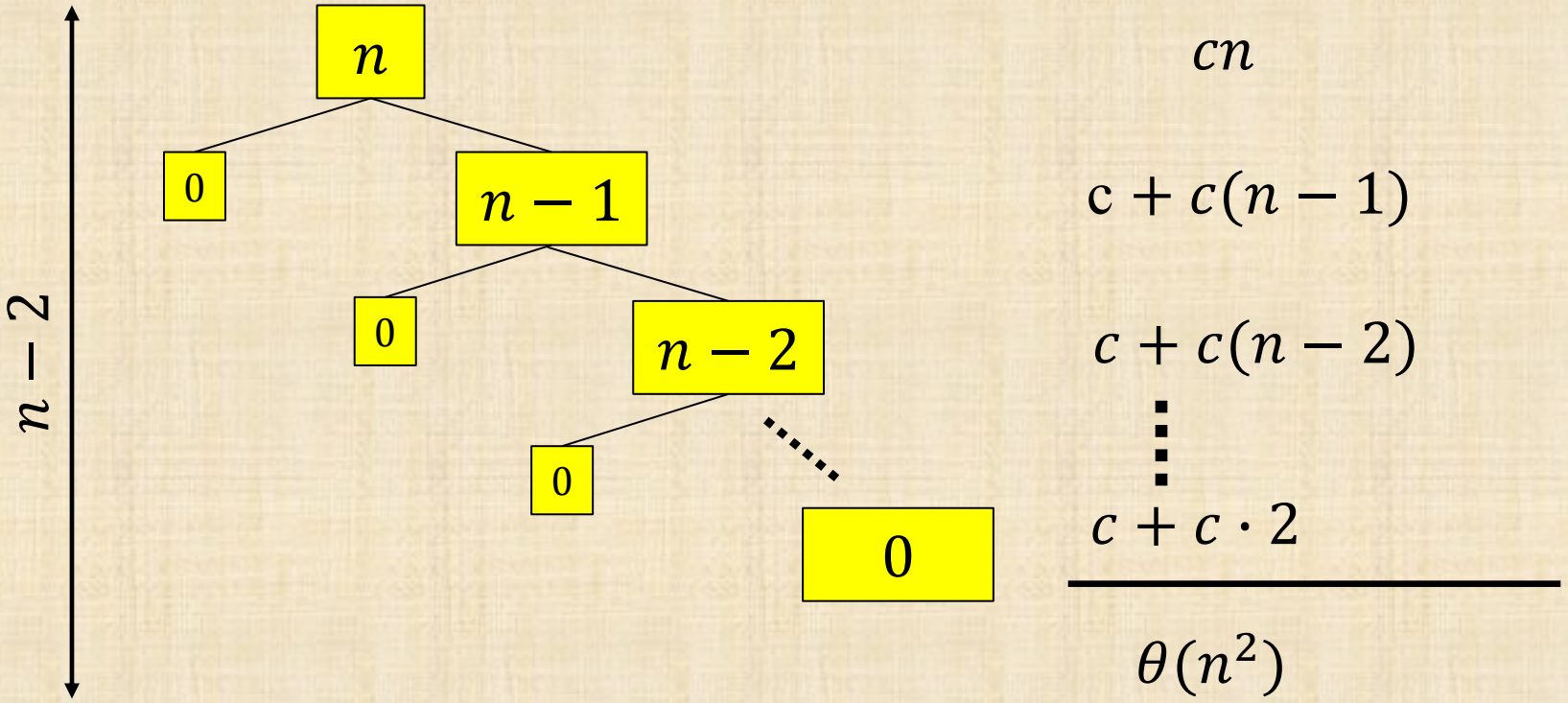


We have already seen that the running time in this case is $\theta(n \log n)$

So $T_{\text{best}}(n) \in \Theta(n \log n)$

QuickSort Worst Case

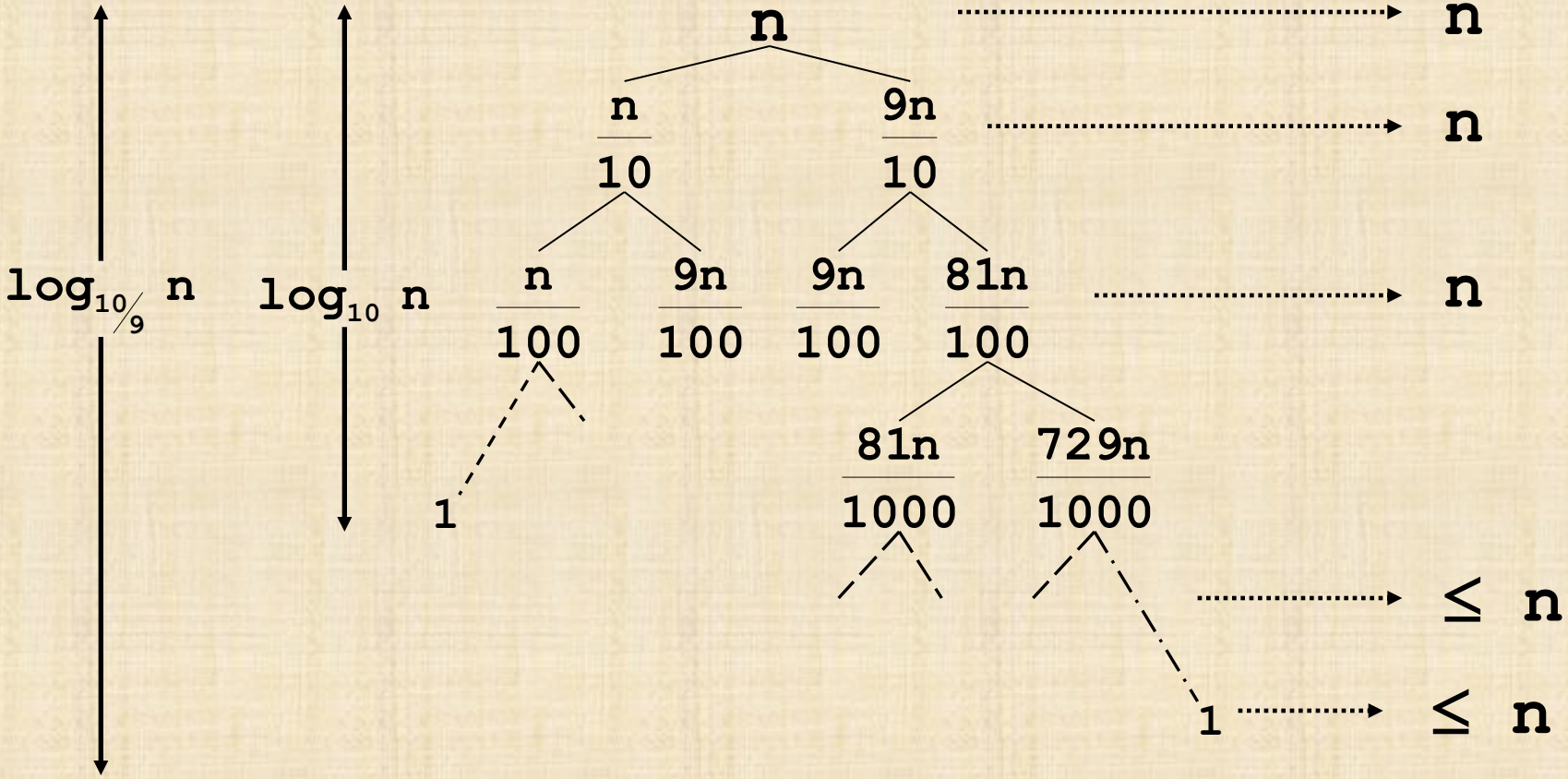
The algorithm always chooses the worst pivot – the one which produces the most unbalanced partition



Picking the Pivot

- Ideal pivot?
  - The median
  - What is the problem?
- Possible choices:
  - The *first* (or the *last)* element
  - The median of leftmost, middle, and rightmost elements
  - Random
    (needs a random number generator)

Balanced Partitioning - meaning



$$T(n) = T\left(\frac{9n}{10}\right) + T\left(\frac{n}{10}\right) + n = \Theta(n \log n)$$

Example for other partitions

- Partition ratio: size of bigger part / size of input

- If the partition ratio is 1/2:
  - The depth of the recursion is: $\log_2 n$
- If the partition ratio is 2/3:
  - The depth of the recursion is: $\log_{3/2} n$
- If the partition ratio is 9/10:
  - The depth of the recursion is: $\log_{10/9} n$
- If the partition ratio is 99/100:
  - The depth of the recursion is: $\log_{100/99} n$
- If the partition ratio is 1/2 every 2 rounds:
  - The depth of the recursion is at most $2 \cdot \log_2 n$

Quicksort Average Case

- Average case is over all possible inputs
- Time complexity: $O(n \log n)$
- Informal intuition:
  - If the partition ratio is some constant $c < 1$, then the depth of the recursion is $\log_{1/c} n = O(\log n)$
  - If the partition ratio is some constant $c < 1$ every constant number d of rounds, then the depth of the recursion is at most $d \cdot \log_{1/c} n = O(\log n)$

Quicksort Average-Case

$$T(n) = d \cdot n + T(p-1) + T(n-p)$$

$$T_{avg}(n) = \sum_{k=1}^{n} \Pr[p=k] \cdot \left(d \cdot n + T_{avg}(k-1) + T_{avg}(n-k) \right)$$

Assuming all inputs are equally likely, the partitioning element (pivot) has probability $\mathbf{1/n}$ to be in each of $\boldsymbol{n}$ final positions

$$T_{avg}(n) = d \cdot n + \frac{1}{n} \cdot \sum_{k=1}^{n} \left(T_{avg}(k-1) + T_{avg}(n-k) \right)$$

Partition

probability

recursion

Quicksort Average-Case

$$T_{avg}(n) = d \cdot n + \frac{2}{n} \cdot \sum_{k=1}^{n} T_{avg}(k-1)$$

Multiply both sides by **n**

$$n \cdot T_{avg}(n) = d \cdot n^2 + 2 \cdot \sum_{k=1}^{n} T_{avg}(k-1)$$

Substitute **n-1** for **n**

$$(n-1) \cdot T_{avg}(n-1) = d(n-1)^2 + 2 \cdot \sum_{k=1}^{n-1} T_{avg}(k-1)$$

Subtract

$$nT_{avg}(n) - (n-1)T_{avg}(n-1) = d \cdot n^2 - d(n-1)^2 + 2T_{avg}(n-1)$$

Quicksort Average-Case

$$nT_{avg}(n)-(n-1)T_{avg}(n-1)=d\cdot n^2-d(n-1)^2+2T_{avg}(n-1)$$

Simplify: $nT_{avg}(n)=(n+1)T_{avg}(n-1)+2dn-d$

Drop the $-d$ to turn $=$ into $\leq$ and

Divide by $n(n+1)$:

$$\frac{T_{avg}(n)}{n+1}\leq\frac{T_{avg}(n-1)}{n}+\frac{2\cdot d}{n+1}$$

$$\frac{T_{avg}(n)}{n+1} \le \frac{T_{avg}(n-1)}{n} + \frac{2d}{n+1}$$

$$S(n) \le S(n-1) + \frac{2d}{n}$$

$$\le S(n-2) + \frac{2d}{n-1} + \frac{2d}{n}$$

$$\le S(n-3) + \frac{2d}{n-2} + \frac{2d}{n-1} + \frac{2d}{n}$$

$$\ldots \le S(1) + \frac{2d}{2} + \frac{2d}{3} + \cdots + \frac{2d}{n}$$

$$< 2d + \frac{2d}{2} + \frac{2d}{3} + \cdots + \frac{2d}{n}$$

$$S(n) \le 2d \sum_{k=1}^{n} \frac{1}{k} = \theta(\log n)$$

Define:

$$S(k) = \frac{T_{avg}(k)}{k+1}$$

$$S(1) = \frac{T_{avg}(1)}{2} = \frac{d}{2} < 2d$$

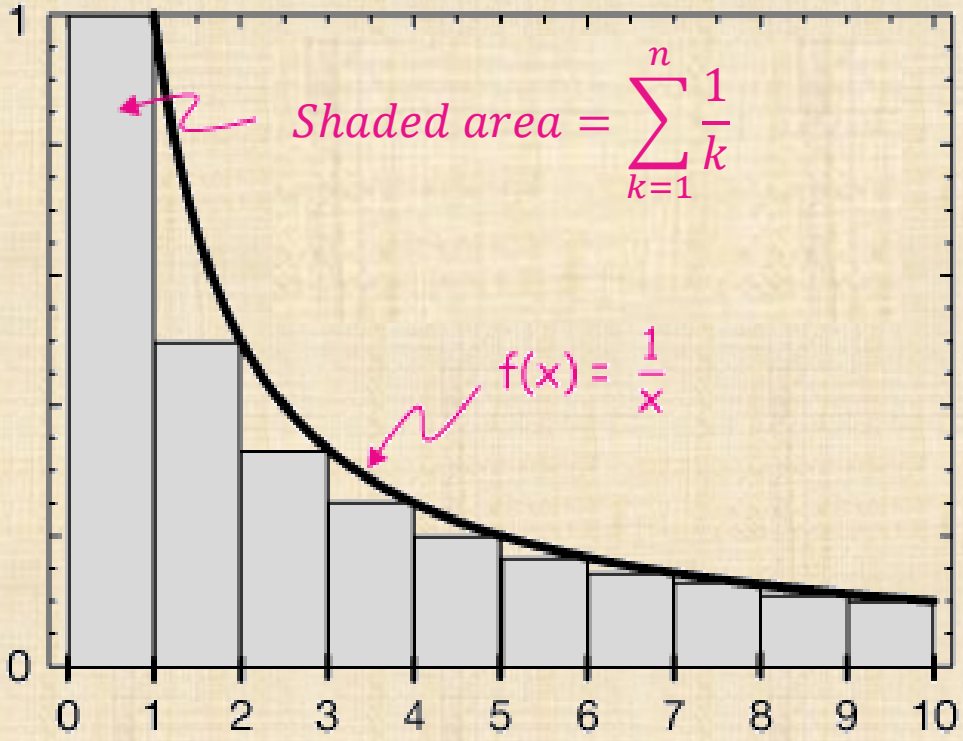
To be completely formal you need an induction

proof on next slide

$H_n = \theta(\log n)$

$$\sum_{k=1}^{n} \frac{1}{k} = 1 + \sum_{k=2}^{n} \frac{1}{k} \leq 1 + \int_1^n \frac{1}{x} dx = 1 + (\ln n - \ln 1) =$$

$$= 1 + (\ln n - 0) = \theta(\log n)$$



Quicksort Average-case

$$S(n) \in \theta(\log n)$$

But $S(n) = \frac{T_{avg}(n)}{n+1}$

And so

$$T_{avg}(n) \in \theta(n \log n)$$

Properties of Quicksort

- Time complexity:
  - Best & average cases: $\Theta(n \log n)$
  - Worst case: $\Theta(n^2)$
- Not stable
- "In-place", but uses auxiliary storage because of recursive calls
  - No true iterative version exists
  - Additional memory worst $\Theta(n)$
    average $\Theta(\log n)$

Properties of Quicksort

- The fastest (in most practical settings) among sorting algorithms that have average time complexity $\Theta(n \log n)$
- Not so fast for small inputs
  - Use early termination

Summary

- Heap Sort:
  - Best/worst/average case $\Theta(n \log n)$
  - In place & Not Stable
- Merge Sort:
  - Best/worst/average cases $\Theta(n \log n)$
  - Stable, not in place
- Quick Sort
  - Worst case $\Theta(n^2)$
  - Best/average cases $\Theta(n \log n)$
  - In place but not immediately stable